

Algorithmen Übungen

HINWEIS ZU DEN UNIT TESTS

Für alle deine Unit Tests gilt:

- 100 % Testabdeckung
- AAA-Pattern
- Mindestens einen zusätzlichen Edge Case
(d. h. du hast dann theoretisch mehr als 100 % Testabdeckung)

Hello World

Schreibe eine main-Methode, die "Hello World" in der Konsole ausgibt.



Hello World – Unit Test

Schreibe deine main-Methode aus der vorherigen Aufgabe "Hello World" um.

Entferne `System.out` aus der Methode und füge stattdessen ein `"return"` hinzu, das `"Hello World"` zurückgibt, um die Methode testen zu können (anschließend ist es keine main-Methode mehr).

Schreibe einen Unit Test für deine überarbeitete Methode.

FizzBuzz

Für Zahlen von 1 bis 100, gebe für jede Zahl die Zahl selbst aus,

- außer wenn die Zahl durch 3 teilbar ist, gebe "Fizz" aus,
- wenn sie durch 5 teilbar ist, gebe "Buzz" aus,
- und wenn sie durch beide Zahlen teilbar ist, gebe "FizzBuzz" aus.

Die Ausgabe soll in der Konsole erfolgen.

Schreibe das Programm in einer main-Methode.

FizzBuzz – Unit Test

Schreibe deine Lösung "FizzBuzz" so um, dass sie (Unit Test) testbar ist.

Schreibe deine Methode außerdem so um, dass man über Parameter den Zahlenbereich ändern kann, d. h. den Startwert und Endwert für die FizzBuzz-Schleife.

Schreibe Unit Tests für deine überarbeitete Version (orientiere dich hierzu an Hello World).

Object Caster

Schreibe eine Methode, die eine Liste mit Objekten erhält. D.h., die Liste kann Objekte vom beliebigen Typ enthalten, z. B. Integer, Double, und String.

Iteriere über die Liste und berechne **für jede Ganzzahl (Integer) das Quadrat**. Gebe die Quadrate als neue Liste zurück:

```
static List<Integer> toSquaredIntegerList(List<Object> objects)
```

Falls ein Objekt nicht gecastet werden kann, soll das Objekt **nicht** in die Ergebnisliste mit aufgenommen werden (es wird ignoriert).

Schreibe einen Unit Test für die Klasse.

Bonusaufgabe:

Schreibe deine Lösung so um, dass sie beliebige Zahlen (Integer, Byte, Short, Long, Float, Double) umwandelt und quadriert. Der Rückgabetyt bleibt hierbei List<Integer>.

Tipp: java.lang.Number

ListConverter

Schreibe eine Klasse ListConverter und implementiere die folgenden Methoden:

// Wandle die gegebene List<Integer> in ein Integer[]-Array um und gebe das Array zurück.

Integer[] **toArray**(List<Integer> list)

// Wandle das gegebene Integer[]-Array in eine List<Integer> um und gebe die List zurück.

List<Integer> **toList**(Integer[] array)

Schreibe einen Unit Test die Klasse.

MyStack<T> mit ArrayList

Implementiere eine eigene **generische** Klasse MyStack<T>, die **intern** eine **ArrayList** verwendet und folgende Methoden besitzt:

// Fügt das element ans Ende des Stacks hinzu.

```
public void push(T element)
```

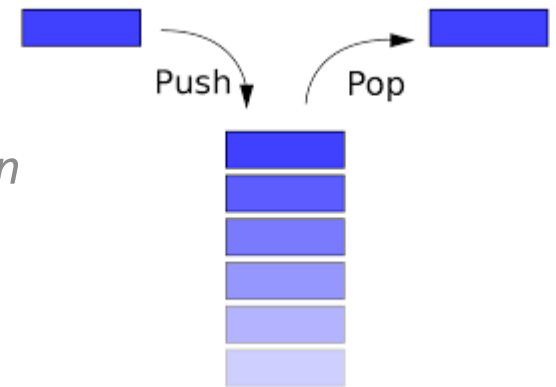
// Entfernt das zuletzt gepushte element aus dem Stack und gibt es zurück.

// Wenn kein Element vorhanden ist, wird eine NoSuchElementException geworfen

```
public T pop() throws NoSuchElementException
```

// true, wenn keine Elemente enthalten sind, sonst false .

```
public boolean isEmpty()
```



Schreibe einen Unit Test für die Klasse MyStack<T>.

Hinweis:

Die Klassen Stack, Deque und LinkedList sind nicht erlaubt für diese Aufgabe.

MyStack<T> mit Array

Schreibe die **generische** Klasse **MyStack<T>** um (aka Refactoring), sodass sie **intern** ein **Object-Array** (Object[]) verwendet, um die Elemente zu speichern.

Falls das interne Array voll ist, soll ein neues Array mit der doppelten Größe angelegt werden.
(ohne Optimierung: das Reduzieren des Arrays muss nicht implementiert werden).

Verwende denselben (deinen) Unit Test von der vorherigen Aufgabe.

Die Klasse hat weiterhin folgende Methoden:

// Fügt das element ans Ende des Stacks hinzu.

```
public void push(T element)
```

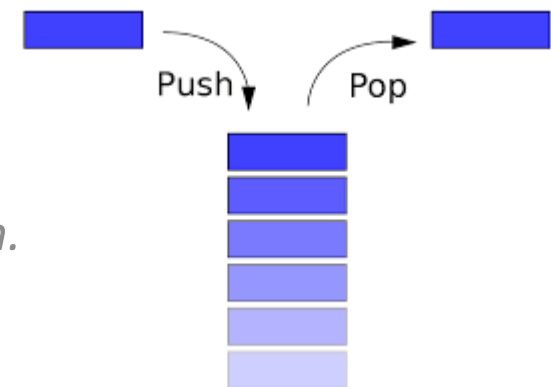
// Entfernt das zuletzt gepushte element aus dem Stack und gibt es zurück.

// Wenn kein Element vorhanden ist, wird eine NoSuchElementException geworfen.

```
public T pop() throws NoSuchElementException
```

// true, wenn keine Elemente enthalten sind, sonst false .

```
public boolean isEmpty()
```



Longest String

Finde in einer Collection den längsten String und gebe ihn zurück.
Falls kein String gefunden wird, gebe einen leeren String "" zurück.

Beispiel:

```
Collection<String> words = List.of("Apple", "Orange", "Strawberry", "Banana");  
String longestString = LongestString.findLongestString(words); // Strawberry
```

Überlege dir, welcher Edge-Case hier wahrscheinlich auftreten könnte.
Schreibe einen Unit Test für deine Lösung.

Linear Search

Implementiere eine Methode, die eine **Collection** von ganzen Zahlen sowie eine gesuchte Zahl entgegennimmt.

Die Methode gibt den **Index** der gesuchten Zahl in der **Collection** zurück, falls die Zahl vorhanden ist, sonst -1.

Bei mehrfachen Vorkommen der gesuchten Zahl, soll der **Index** der zuerst gefundenen Zahl zurückgegeben werden.

Tipp: Es könnte hilfreich sein, die empfangene Collection in eine Liste umzuwandeln.

Beispiel:

```
Collection<Integer> numbers = List.of(1, 2, 3, 4, 5);  
int index = LinearSearch.findIndexOfNumber(numbers, 3); // 2
```

Schreibe einen Unit Test für deine Lösung.

Find MinMax

Implementiere eine Methode, die eine Collection von Ganzzahlen entgegennimmt und ein **record** MinMax mit zwei Attributen zurückgibt: dem kleinsten und dem größten **Wert** in der Eingabe-Collection.

Falls nur ein Element vorhanden ist, ist dieses Element zugleich Minimum und Maximum.

Wenn keine gültigen Werte gefunden werden, wird eine **NoSuchElementException** geworfen.

Beispiel:

```
Collection<Integer> integers = List.of(1, 2, 3, 4, 5, 0);  
MinMax minMax = MinMaxUtil.findMinMax(integers); // MinMax[min: 0, max: 5]
```

Schreibe einen Unit Test für deine Lösung.